



Formación

Curso Fundamentos Desarrollo Java Con Spring Framework

indra



Formador

Ana Isabel Vegas



INGENIERA INFORMÁTICA con Master Máster Universitario en Gestión y Análisis de Grandes Volúmenes de Datos: Big Data. Certificada en PCEP en Lenguaje de Programación Python, PCAP Python Certified Associate Programmer y la certificación JSE en Javascript. Desarrolladora de Aplicaciones FULLSTACK, se dedica desde hace + de 20 años a la CONSULTORÍA y FORMACIÓN en tecnologías del área de DESARROLLO y PROGRAMACIÓN.

Experta en lenguajes de programación, herramientas y tecnologías tanto del lado del servidor como del lado del cliente, imparte formación dirigida especialmente a profesionales dedicados al desarrollo de aplicaciones y soluciones, tanto en la parte BACK (Tecnologías: JAVA, Spring -MVC, Core, Batch, Webflow, Security, Microservicios, Spring Boot-, Hibernate/JPA, JSF, JSF, AJAX, JQuery, Maven, Git, SVN, Gradle además de impartir formación en Python, NodeJS, R, BigData, Machine Learning, arquitectura de microservicios, programación reactiva y Servidores de Aplicaciones), como en la parte FRONT (Tecnologías: JavaScript, TypeScript, Angular JS/Angular v2 yss, React, Reux, CSS, HTML, Bootstrap, entre otras).



training@iconotc.com

❑ **Duración:** 25 horas

❑ **Modalidad:** On-line

❑ **Fechas/Horario:**

- Días 8, 9, 10, 11, 15, 16, 17 y 18 Junio 2026
- Horario 15:00 – 17:30 hs.

❑ **Contenidos:**

- Introducción al framework
 - Principios y arquitectura
 - BeanFactory y ApplicationContext
- Principio de Inversión de control (IoC) / Inyección de Dependencias (DI)
 - DI por propiedades
 - DI por constructores
 - Tipos complejos de dependencias
 - Importar ficheros de configuración
 - Espacios de nombres P y C
 - Métodos de inicialización/destrucción
 - Oyentes de eventos
 - Autowiring
 - Spring Annotation Config
 - Spring Java Config
 - Spring Expression Language (SpEL)
- Programación orientada a aspectos (AOP)
 - Proxies Dinamicos vs CGLIB
 - Advices y Advisors
 - Tipos de aspectos
 - AutoProxies
 - Espacio de nombres AOP y AspectJ
- Uso de Spring para acceso a Bases de Datos
 - Uso de DataSources y BD embebidas
 - JdbcTemplate y SimpleJdbcTemplate
 - HibernateTemplate y JpaTemplate
 - Spring Data JPA
- Control de Transacciones en Spring
 - Propiedades de las transacciones
 - Uso del TransactionProxyFactoryBean
 - Uso programático de transacciones
 - El espacio de nombres TX y Anotaciones
 - Creación de tests transaccionales

Introducción al framework

Tema 1

Rod Johnson .-

Rod Johnson

Rod Johnson is an enterprise Java architect specializing in scalable web applications. He has worked with both Java and J2EE since their release, and he is a member of JSR 154 Expert Group defining the Servlet 2.4 specification.



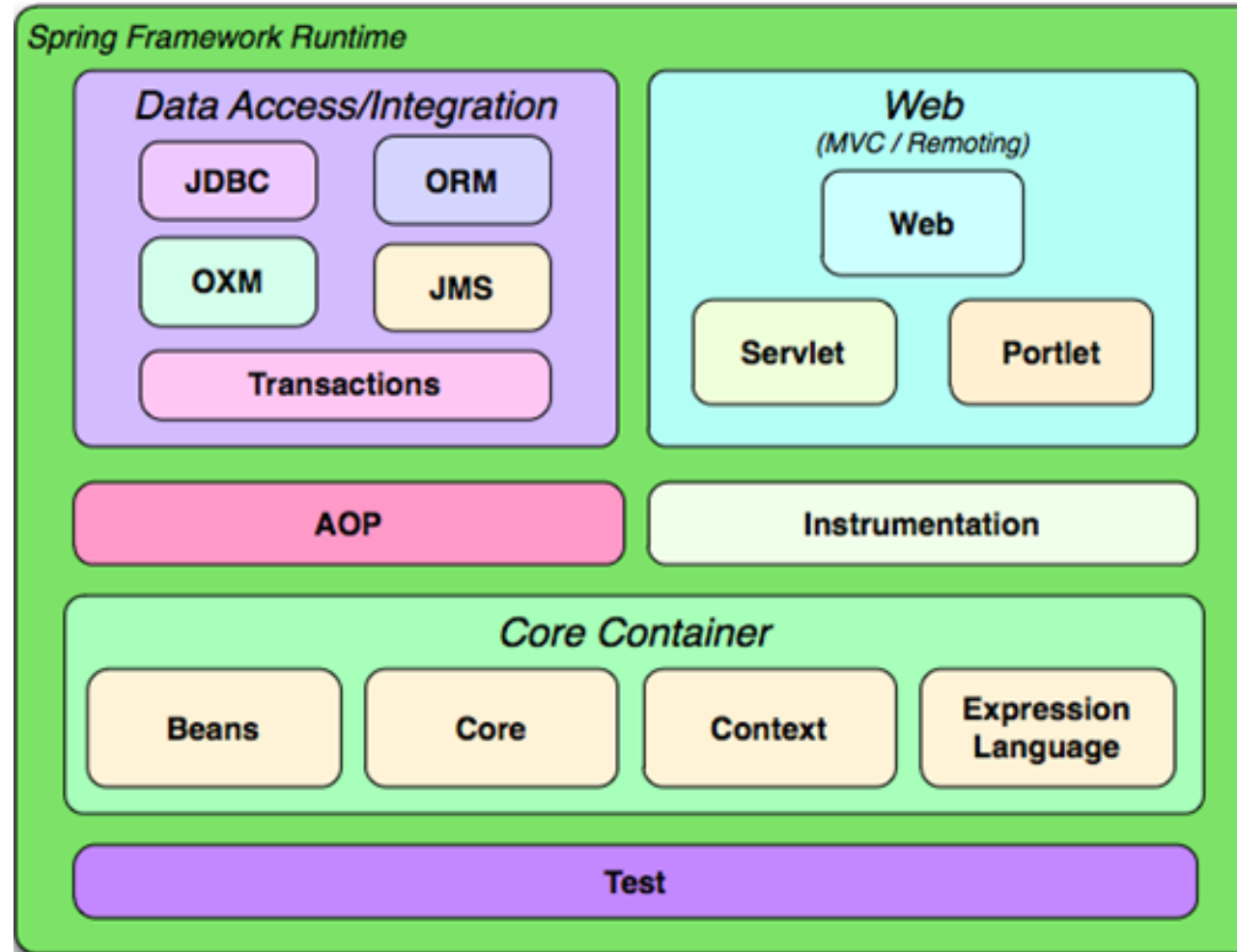
Que es Spring .-

- Spring es un framework de código abierto.
- Fue creado para simplificar el desarrollo de aplicaciones empresariales usando POJO's para conseguir objetivos que antes sólo eran posibles con EJB.
- Es un framework no intrusivo, lo que genera componentes no dependientes del propio marco de trabajo.

Versiones .-

- Primera versión en Junio 2003 fue lanzado bajo la licencia de Apache 2.0
- Spring 1.0 en Marzo 2004
- Spring 2.0 en Octubre 2006
- Spring 2.5 en Noviembre 2007
- Spring 3.0 en Diciembre de 2009
- Spring 4.0 en Diciembre de 2013
- Spring 5.0 en Octubre de 2017

Estructura del Framework .-



Modulos del Framework .-

- Spring está organizado en aproximadamente 20 módulos agrupados en:
 - **Core Container**
 - **Data Access/Integration**
 - **Web**
 - **AOP and Instrumentation**
 - **Test**

Core Container .-

- Modulo **Core**; es el núcleo de Spring
- Modulo **Beans**; junto al modulo Core recoge todos los fundamentos de Spring, incluyendo IoC y DI
- Modulo **Context**; añade soporte para la internacionalización, propagación de eventos, carga de recursos
- Modulo **Expression Language**; lenguaje de expresiones de Spring.

Data Access/Integration .-

- Modulo **JDBC**; Acceso a base de datos mediante JDBC
- Modulo **ORM**; Acceso a base de datos a través JPA, Hibernate, Ibatis, ..etc.
- Modulo **OXM**; soporta el mapeo Objeto/XML como por ejemplo JAXB.
- Modulo **JMS**; Producir y consumir mensajes asíncronos.
- Modulo **Transaction**; Para manejar transacciones programáticas o declarativas.

Web .-

- Modulo **Web**; Funcionalidades básicas de aplicaciones web.
- Modulo **Web-Servlet**; mas conocido como **Spring MVC**
- Modulo **Web-Struts**; permite integrar Struts 2.0 con Spring 3.0.
- Modulo **Web-Portlets**; integración de Portlets.

AOP .-

- Modulo **AOP**; Programación Orientada a Aspectos.

Instrumentation .-

- Modulo **Instrumentation**; implementación del cargador de clases para ser utilizado en determinados servidores de aplicaciones.

Test .-

- Modulo **Test**; permite probar componentes Spring con Junit.

Descarga de módulos .-

- Para poder utilizar un modulo de Spring es necesario descargarlo de la Web.
- El formato de los módulos será el siguiente:

`org.springframework.modulo-version.jar`

- La relación de módulos es la siguiente:

Descarga de módulos .-

- org.springframework.aop
- org.springframework.beans
- org.springframework.context
- org.springframework.context.support
- org.springframework.expression
- org.springframework.instrument
- org.springframework.jdbc
- org.springframework.jms

Descarga de módulos .-

- org.springframework.orm
- org.springframework.oxm
- org.springframework.test
- org.springframework.transaction
- org.springframework.web
- org.springframework.web.portlet
- org.springframework.web.servlet
- org.springframework.web.struts

Principio de Inversión de control (IoC) / Inyección de Dependencias (DI)

Tema 2

Contenedores de bean .-

- En una aplicación desarrollada con Spring, los bean (objetos de aplicación) van a residir dentro del contenedor de beans.
- Dicho contenedor se encuentra en el núcleo (Core).
- El contenedor utiliza DI (Inyección de Dependencias) para crear los beans, conectarlos con otros, configurarlos y administrar su ciclo de vida.

Tipos de contenedores .-

- BeanFactory; Son los contenedores mas sencillos, proporcionan un soporte básico para DI y se utilizan para recursos limitados, p.e. móviles
- ApplicationContext; Son una especificación de BeanFactory (hereda de esta). Esta clase llama al contenedor de beans a partir de un XML.

ApplicationContext .-

- Para crear el contexto de aplicación, lo podemos hacer por medio de 3 clases:
 - `ClassPathXmlApplicationContext`; carga la definición de contexto a partir de un archivo XML situado en el classpath.
 - `FileSystemXmlApplicationContext`; carga la definición de contexto a partir de un archivo XML situado en el sistema de archivos.
 - `XmlWebApplicationContext`; carga la definición de contexto a partir de un archivo XML situado en una aplicación web.

Configuración de beans mediante XML.-

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

    <!-- Declaraciones de beans -->

</beans>
```

Formas de crear un bean .-

- Podemos usar diferentes alternativas a la hora de crear un bean :
 - A través de constructor
 - Mediante una factoría estática
 - Utilizando una factoría de instancia

A través de constructor .-

- Constructor sin argumentos:

```
<bean id="miBean" class="app.ClienteBean"/>
```

Es lo mismo que:

```
ClienteBean miBean = new ClienteBean();
```

A través de constructor .-

- Constructor con argumentos:

```
<bean id="direccion" class="app.DireccionBean"/>
<bean id="miBean" class="app.ClienteBean">
    <constructor-arg value="Juan" />
    <constructor-arg ref="direccion" />
</bean>
```

Es lo mismo que:

```
DireccionBean dir = new DireccionBean();
ClienteBean miBean = new ClienteBean("Juan", dir);
```

A través de constructor .-

```
<bean id="exampleBean" class="examples.ExampleBean">  
  <constructor-arg type="int" value="7500000"/>  
  <constructor-arg type="java.lang.String" value="42"/>  
</bean>
```

```
<bean id="exampleBean" class="examples.ExampleBean">  
  <constructor-arg index="0" value="7500000"/>  
  <constructor-arg index="1" value="42"/>  
</bean>
```

```
<bean id="exampleBean" class="examples.ExampleBean">  
  <constructor-arg name="years" value="7500000"/>  
  <constructor-arg name="ultimateanswer" value="42"/>  
</bean>
```

Mediante una factoría estática .-

```
<bean id="miBean"  
      class="app.ClienteFactory"  
      factory-method="crearBean"/>
```

Es lo mismo que:

```
miBean = ClienteFactory.crearBean();
```

Utilizando una factoría de instancia

```
<bean id="miFactory" class="app.Factoria" />  
<bean id="miBean"  
    factory-bean="miFactory"  
    factory-method="crearBean"/>
```

Es lo mismo que:

```
Factoria miFactory = new Factoria();  
miBean = miFactory .crearBean();
```

Inyección de propiedades de un bean .-

- Beans externos:

```
<bean id="taller" class="negocio.TallerCarroceria" />
<bean id="seguro" class="negocio.SeguroCoche" scope="prototype" >
    <property name="taller" ref="taller"/>
    <property name="email" value="email@dom.com"/>
</bean>
```

- Beans internos:

```
<bean id="seguro" class="negocio.SeguroCoche">
    <property name="taller">
        <bean class="negocio.TallerCarroceria" />
    </property>
    <property name="email" value="email@dom.com"/>
</bean>
```

Colecciones .-

```
<bean id="moreComplexObject" class="example.ComplexObject">
  <!-- results in a setAdminEmails(java.util.Properties) call -->
  <property name="adminEmails">
    <props>
      <prop key="administrator">administrator@example.org</prop>
      <prop key="support">support@example.org</prop>
      <prop key="development">development@example.org</prop>
    </props>
  </property>
  <!-- results in a setSomeList(java.util.List) call -->
  <property name="someList">
    <list>
      <value>a list element followed by a reference</value>
      <ref bean="myDataSource" />
    </list>
  </property>
  <!-- results in a setSomeMap(java.util.Map) call -->
  <property name="someMap">
    <map>
      <entry key="an entry" value="just some string"/>
      <entry key="a ref" value-ref="myDataSource"/>
    </map>
  </property>
  <!-- results in a setSomeSet(java.util.Set) call -->
  <property name="someSet">
    <set>
      <value>just some string</value>
      <ref bean="myDataSource" />
    </set>
  </property>
</bean>
```

```
bean | ref | idref | list | set | map | props | value | null
```

List .-

Al igual que utilizamos la etiqueta <ref> también podemos utilizar <value>, <bean> y <null>

```
<bean id="guitarra" class="negocio.Guitarra"/>
<bean id="piano" class="negocio.Piano"/>
<bean id="bateria" class="negocio.Bateria"/>

<bean id="orquesta" class="negocio.Orquesta">
  <property name="instrumentos">
    <list>
      <ref bean="guitarra"/>
      <ref bean="piano"/>
      <ref bean="bateria"/>
    </list>
  </property>
</bean>
```

Set .-

Al igual que List lo podemos utilizar para definir un array.
Podemos utilizar <value>, <bean> y <null>

```
<bean id="guitarra" class="negocio.Guitarra"/>
<bean id="piano" class="negocio.Piano"/>
<bean id="bateria" class="negocio.Bateria"/>

<bean id="orquesta" class="negocio.Orquesta">
    <property name="instrumentos">
        <set>
            <ref bean="guitarra"/>
            <ref bean="piano"/>
            <ref bean="bateria"/>
        </set>
    </property>
</bean>
```

Map .-

En lugar de <key> podemos utilizar <key-ref> y en lugar de <value> la alternativa es <value-ref>

```
<bean id="guitarra" class="negocio.Guitarra"/>
<bean id="piano" class="negocio.Piano"/>
<bean id="bateria" class="negocio.Bateria"/>

<bean id="orquesta" class="negocio.Orquesta">
    <property name="instrumentos">
        <map>
            <entry key="clave1" value-ref="guitarra"/>
            <entry key="clave2" value-ref="piano"/>
            <entry key="clave3" value-ref="bateria"/>
        </map>
    </property>
</bean>
```

Properties .-

La clase Properties es prácticamente lo mismo que un Map con la excepción que solo soporta tipos String para claves y valores.

```
<bean id="orquesta" class="negocio.Orquesta">
  <property name="instrumentos">
    <props>
      <prop key="clave1">Guitarra</prop>
      <prop key="clave2">Piano</prop>
      <prop key="clave3">Bateria</prop>
    </props>
  </property>
</bean>
```

Tipos de ámbito de un bean .-

- Singleton; Por defecto todos los beans son instancias únicas.
- Prototype; Permite que un bean se instancie tantas veces se quiera.
- Request; El bean pertenece al ámbito de petición. Solo para aplicaciones Web.
- Session; El bean pertenece al ámbito de sesión. Solo para aplicaciones Web.
- Global-session; El bean pertenece al ámbito de sesión global. Solo es válido para el contexto de un portlets.

Definir el ámbito de un bean .-

El siguiente fragmento implica que cada vez que se utilice el método `getBean()` para recuperar el bean “seguro” del contenedor, se crea una nueva instancia de este.

```
<bean id="seguro" class="negocio.SeguroCoche" scope="prototype">
    <property name="taller">
        <bean class="negocio.TallerCarroceria" />
    </property>
    <property name="email" value="email@dom.com" />
</bean>
```

Varios archivos XML.-

- Podemos separar los ficheros de configuración y que los lea automáticamente el contenedor utilizando la etiqueta <import>

```
<import resource="servicios.xml"/>
```

- Para cargar el contenedor de beans lo haremos a partir del XML principal.

```
ApplicationContext factoria =  
    new ClassPathXmlApplicationContext("spring.xml");
```

Herencia de beans.-

Los beans se pueden configurar con herencia en el xml y de esta forma reutilizar sus propiedades.

```
<!-- el bean padre no se puede recuperar del contenedor ya que es abstracto -->
<bean id="padre" class="es.prac.spring.ObjetoValores" abstract="true" >
    <property name="codigo" value="1" />
    <property name="nombre" value="nombre generico" />
</bean>
<!-- hijo1 hereda de padre y se cambia el valor de su propiedad nombre -->
<bean id="hijo1" parent="padre">
    <property name="nombre" value="nombre concreto" />
</bean>
<!-- hijo2 hereda de padre tomara las propiedades de padre por defecto -->
<bean id="hijo2" parent="padre" />
```

Metodos init y destroy.-

- En la clase creamos un método init y otro destroy

```
public class BeanInicializacionMetodos {  
  
    public void inicializa() {  
        System.out.println("BeanInicializacionMetodos:: Metodo de inicializacion");  
    }  
  
    public void libera() {  
        System.out.println("BeanInicializacionMetodos:: Metodo de destruccion");  
    }  
}
```

- Configuración en el xml

```
<bean id="beanDos" class="es.prac.spring.BeanInicializacionMetodos"  
      init-method="inicializa" destroy-method="libera" />
```

Metodos init y destroy por defecto.-

- Si en todas las clases creamos un método iniciar() y otro destruir() estos se pueden configurar de forma conjunta.

```
<beans xmlns="http://www.springframework.org/schema/beans"  
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
       xsi:schemaLocation="http://www.springframework.org/schema/beans  
                           http://www.springframework.org/schema/beans/spring-beans-3.0.xsd"  
       default-init-method="iniciar" default-destroy-method="destruir">
```

- Es importante saber que en los beans declarados como prototype no se invoca al método destroy.

InitializingBean y DisposableBean.-

```
import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;

public class BeanInicIALIZACIONInterfaces implements InitializingBean,
                                                    DisposableBean {
    // Este metodo los implementamos de la interfaz InitializingBean
    public void afterPropertiesSet() throws Exception {
        System.out.println("Propiedades inyectadas");
    }

    // Este metodo los implementamos de la interfaz DisposableBean
    public void destroy() throws Exception {
        System.out.println("El bean se va a destruir");
    }
}
```

Crear beans de forma lazy.-

- Si ponemos el atributo lazy-init a true, el bean es creado al invocarlo por primera vez. Su valor por defecto es false.
- Es importante saber que esta propiedad no se hereda. Se debe definir específicamente para cada bean.

```
<bean id="servicioHola" class="es.prac.spring.HolaMundo"  
      lazy-init="true" />
```

registerShutdownHook.-

- Si creamos el ApplicationContext como AbstractApplicationContext podemos invocar al metodo registerShutdownHook para indicar que al finalizar la aplicación llame a todos los destroy.

```
AbstractApplicationContext factoria =  
    new ClassPathXmlApplicationContext("spring.xml");  
  
/* Si no se invoca el metodo registerShutdownHook de la factoria  
 * no se invocan a los destroy de los singleton,  
 * para los prototypes no se invoca a los destroy*/  
factoria.registerShutdownHook();
```

Espacio de nombres p.-

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:p="http://www.springframework.org/schema/p"

       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

    <bean id="direccionBean" class="app.modelo.Direccion"
          p:calle="Mayor"
          p:poblacion="Madrid"
          p:cp="28014" />

    <bean id="clienteBean" class="app.modelo.Cliente"
          p:nombre="Juan"
          p:direccion-ref="direccionBean" />

</beans>
```

Conexión automática con autowire.-

- Podemos efectuar conexiones automáticas de un bean con sus dependencias de 4 formas:
 - `byName`; una propiedad se conecta con un bean que tenga el mismo nombre de esta.
 - `byType`; una propiedad se conecta con un bean que sea del mismo tipo que esta.
 - `constructor`; busca los beans que coinciden con los argumentos del constructor. Es igual que `byType` pero con constructores en vez de propiedades
 - `autodetect`; primero lo intenta con constructor y si no puede lo intenta por tipo (`byType`)

Ejemplo.-

```
<bean id="taller" class="negocio.TallerMecanica"/>

<bean id="seguro" class="negocio.SeguroCoche" autowire="byType">
    <!-- <property name="taller" ref="taller" /> -->
    <property name="email" value="email@dom.com"/>
</bean>
```

Conexión automática predeterminada.-

- Podemos configurar para que todos los beans del contenedor se conecten de forma automática utilizando el mismo estilo de conexión.

```
<beans xmlns="http://www.springframework.org/schema/beans"  
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
       xsi:schemaLocation="http://www.springframework.org/schema/beans  
                           http://www.springframework.org/schema/beans/spring-beans-3.0.xsd"  
  
       default-autowire="byType">
```

Excluir un bean de conexión automática.-

- Si queremos excluir aquellos beans que no queremos que se tengan en cuenta para la conexión automática,

```
<bean id="taller" class="negocio.TallerMecanica"  
      autowire-candidate="false" />
```

Combinar conexiones automáticas y explicitas.-

- Podemos seguir utilizando conjuntamente autowire y también inyectar una propiedad con <property>
- Sin embargo no podemos combinar la conexión automática en los constructores con elementos <constructor-arg>

Conexión con anotaciones.-

- Para poder utilizar anotaciones para realizar conexiones automáticas necesitamos activarlo mediante el espacio de nombres context.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context-3.0.xsd">

    <context:annotation-config />

</beans>
```

@Autowired.-

- En el siguiente método Spring intenta la conexión automática por tipo, esto es, si encuentra un bean de tipo Instrument lo conecta.

```
@Autowired
public Instrumentalist(Instrument instrument) {
    this.instrument = instrument;
}
```

- Podemos utilizar esta anotación en constructores, propiedades y cualquier método aunque sean privados.
- Si no existe un bean que poder conectar o hay varios, Spring genera NoSuchBeanDefinitionException.

Conexión opcional.-

- El atributo `required` indica que la propiedad se puede conectar o no, en este segundo caso tomará el valor `null`.

```
@Autowired(required=false)  
private Instrument instrument;
```

- Con sobrecarga de constructores, solo un constructor se puede configurar como `required=true` el resto deben ser `false`, Spring selecciona el constructor que tenga mas argumentos que puedan cumplirse.

Dependencias ambiguas.-

- Spring intenta conectar de forma automática con un bean cuyo id sea guitar.

```
@Autowired
@Qualifier("guitar")
private Instrument instrument;
```

- Calificador aplicado al propio bean

```
<bean class="com.springinaction.springidol.Guitar" >
    <qualifier value="stringed" />
</bean>
```

```
@Autowired
@Qualifier("stringed")
public class Guitar implements Instrument{
```

@Inject.-

- Otro modelo DI es el propuesto por JSR.330 (también conocido como at inject).
- La anotación @Inject es igual que @Autowired de Spring con la excepción que no tiene un atributo required.

```
@Inject  
private Instrument instrument;
```

Dependencias ambiguas.-

- Al igual que en Spring teníamos @Qualifier, JSR.330 nos proporciona la anotación @Named

```
@Inject  
@Named("guitar")  
private Instrument instrument;
```

Injectar valores con anotaciones.-

- Con la anotación @Value (nueva en Spring 3.0) podemos conectar valores a una propiedad.

```
@Value("Pedro")  
private String nombre;
```

```
@Value("#{cliente.nombre}")  
private String nombre;
```

Detectar los beans de forma automática.-

- Todas las clases que estén anotadas con:
 - @Component; componente de Spring
 - @Controller; controlador MVC
 - @Repository; repositorio de datos
 - @Service; servicio
 - @Configuration; clase que define beans
- Pueden registrarse de forma automática como beans.

Detectar los beans de forma automática.-

- La configuración en el XML es la siguiente:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context-3.0.xsd">

    <context:component-scan base-package="paquete,paquete" />

</beans>
```

Declarar un componente.-

```
@Component("miCliente")
public class Cliente implements Persona{
    // .....
}
```

- El bean se creará con el ID “miCliente”, si no hubiésemos puesto nada el bean se crearía con el nombre de la clase en minúsculas.

Filtros para component-scan.-

- Tenemos los siguientes tipos:
 - annotation; clases con la anotación que indica el atributo expression.
 - assignable; clases que se podrían asignar a lo que indica el atributo expression.
 - aspectj; clases que coinciden con la expresion Aspectj especificada en el atributo expression
 - custom; usa una implementacion TypeFilter especificada en el atributo expression
 - regex; clases cuyo nombre coincide con la expresion regular especificada en el atributo expression

Filtros para component-scan.-

- Este fragmento indica que se registran todas las implementaciones de Persona excepto las que están anotadas con @Service

```
<context:component-scan base-package="paquete, paquete" >
  <context:include-filter type="assignable"
    expression="app.modelo.Persona" />
  <context:exclude-filter type="annotation"
    expression="org.springframework.stereotype.Service" />
</context:component-scan>
```

Definir una clase de configuración.-

```
@Configuration
public class BeansConfig {

    // <bean id="pedro" class="app.modelo.Cliente" />
    @Bean
    public Cliente pedro(){
        return new Cliente();
    }

    // <bean id="antonio" class="app.modelo.Alumno:">
    //      <property name="edad" value="29" />
    //</bean>
    @Bean
    public Alumno antonio(){
        return new Alumno(29);
    }

}
```

Lenguaje de expresiones SpEL.-

- Nuevo a partir de la versión 3 de Spring.
- Podemos inyectar valores literales como:
 - Números:
 - Enteros `#{8}`
 - Decimales `#{8.37}`
 - Notación científica `#{1e5}`
 - Cadenas
 - Comillas simples `#{'texto'}`
 - Comillas dobles `#{“texto”}`
 - Booleanos
 - `#{true}` o `#{false}`

Lenguaje de expresiones SpEL.-

- Referenciar un bean:

```
<property name="direccion" value="#{direccion}" />
```

- Acceder a una propiedad de otro bean:

```
<property name="direccion"  
            value="#{cliente.direccion}" />
```

- Método de otro bean:

```
<property name="direccion"  
            value="#{cliente.direccion()}" />
```

Lenguaje de expresiones SpEL.-

- Acceso a variables estáticas:

```
<property name="numero"  
    value="#{T(java.lang.Math).PI}" />
```

- Acceso a métodos estáticos:

```
<property name="numero"  
    value="#{T(java.lang.Math).random()}" />
```

Lenguaje de expresiones SpEL.-

- Operadores:

- Aritméticos: +, -, *, /, %, ^(potencia)
- Relacionales: <, >, ==, <=, >=, lt, gt, eq, le, ge
- Lógicos: and, or, not o !
- Condicionales:
 - Ternario ?:

```
<property name="numero"  
    value="#{ condicion ? Valor_true : valor_false}" />
```
 - Elvis ?:

```
<property name="numero"  
    value="#{ expresion ?: valor_si_expresion_igual_null}" />
```
- Expresiones regulares: matches

Lenguaje de expresiones SpEL.-

- Acceder a un miembro de una colección:

```
<property name="numero"  
    value="#{numeros[3]}" />
```

- Acceder a un elemento de un mapa por su clave:

```
<property name="propiedad"  
    value="#{mapa['clave']}" />
```

Lenguaje de expresiones SpEL.-

- `systemEnvironment` contiene las variables de entorno del equipo:

```
<property name="homePath"  
    value="#{systemEnvironment['HOME']}" />
```

- `systemProperties` contiene las propiedades introducidas con `-D`:

```
<property name="homePath"  
    value="#{systemProperties['autor']}" />
```

Lenguaje de expresiones SpEL.-

- Seleccionar miembros de la colección en base a una condición: (Selección)

```
<property name="prop"  
    value="#{colección.[condicion]}" />
```

- Primer elemento coincidente de la colección:

```
<property name="prop"  
    value="#{colección.^[condicion]}" />
```

- Ultimo elemento coincidente de la colección:

```
<property name="prop"  
    value="#{colección.$[condicion]}" />
```

Lenguaje de expresiones SpEL.-

- Recopilar elementos de una colección para incluirla en otra con una sola propiedad: (Proyección)

```
<property name="nuevaColeccion"  
    value="#{colección.![propiedad]}" />
```

- Recopilar elementos de una colección para incluirla en otra con varias propiedades:

```
<property name="nuevaColeccion"  
    value="#{colección.![prop1 + ' , ' + prop2]}"  
/>
```

Lenguaje de expresiones SpEL.-

```
<property name="prueba"  
    value="#{coleccion.[condicion].[prop1 + ', ' + prop2]}" />
```

Programación Orientada a Aspectos (AOP)

Tema 3

Definición de aspecto .-

- Un aspecto es una funcionalidad que en un momento dado puede intervenir en la ejecución de un método.

Terminología AOP .-

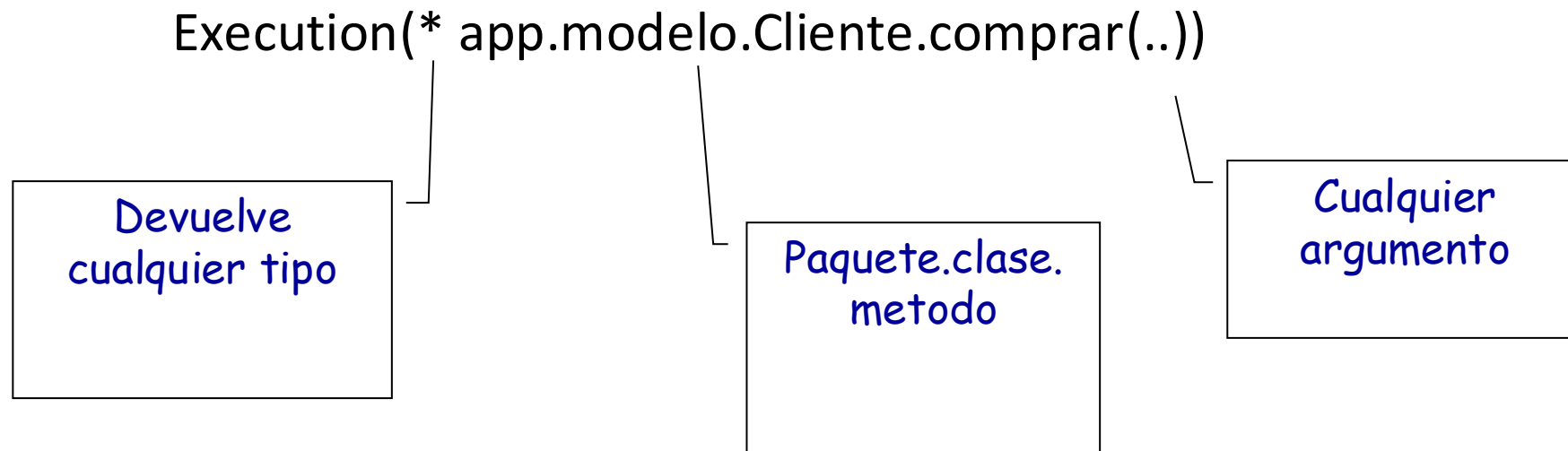
- Advice (consejo); Define el objetivo de un aspecto, su comportamiento y cuando se aplica.
- Joinpoints (puntos de cruce); Son todos los puntos dentro del flujo de ejecución en los que puede aplicarse el consejo (propiedades, métodos, constructores). En Spring AOP solo podemos utilizar métodos.
- Pointcuts (Puntos de corte); Define exactamente donde se aplica el consejo.

Tipos de consejos .-

- **Antes; <aop:before>** el consejo se ejecuta antes del metodo
- **Después; <aop:after>** el consejo se ejecuta después del método. Actúa como finally
- **Despues de la devolución; <aop:after-returning>** el consejo se ejecuta después de que el metodo se complete con éxito
- **Despues de producir; <aop:after-throwing>** el consejo se ejecuta después de que el metodo produzca un error. Funciona como catch.
- **Alrededor; <aop:around>** El consejo encapsula el método, se ejecuta antes y después de este.

AspectJ .-

- En Spring AOP, los puntos de corte se definen usando el lenguaje de expresión AspectJ.
- Usaremos la siguiente sintaxis:



Configurar un aspecto .-

```
<aop:config>
  <aop:pointcut id="performance"
    expression="execution(* app.Performer.actuar(..))"/>
  <aop:aspect ref="publico">
    <aop:before pointcut-ref="performance"
      method="apagarMoviles"/>

    <aop:before pointcut-ref="performance"
      method="tomarAsientos"/>

    <aop:after-returning pointcut-ref="performance"
      method="aplaudir"/>

    <aop:after-throwing pointcut-ref="performance"
      method="exigirDevolucion"/>
  </aop:aspect>
</aop:config>
```

Consejos around .-

```
public void decide(ProceedingJoinPoint jp) throws Throwable {
    System.out.println("## Around ANTES:: Se ha solicitado un pedido ##" +
        jp.getTarget().getClass());

    Object ret = jp.proceed(); // ejecuta el metodo aconsejado

    System.out.println("## Around DESPUES:: Se ha solicitado un pedido ##"
        + ret);
}

<aop:config>
    <aop:pointcut id="referencia"
        expression="execution(* es.prac.spring.PedidoServicio.*(..))" />

    <aop:aspect ref="loggerAround">
        <aop:around pointcut-ref="referencia" method="decide" />
    </aop:aspect>

</aop:config>
```

Paso de parámetros .-

```
<aop:config>
  <aop:pointcut id="referencia"
    expression="execution(* es.prac.spring.PedidoServicio.*(..)) and args(codPedido)" />

  <aop:aspect ref="loggerAround">
    <aop:around pointcut-ref="referencia" method="decide"
      arg-names="jp,codPedido"/>
  </aop:aspect>
</aop:config>
```

Configuración con anotaciones .-

- Las anotaciones forman parte de @AspectJ y me permiten crear aspectos sin necesidad de XML.
- Para que Spring reconozca las anotaciones debemos incluir el siguiente elemento en el XML.

```
<aop:aspectj-autoproxy />
```

@Aspect .-

- Esta anotación marca un POJO como aspecto.
- Se pone a nivel de clase

```
@Aspect  
public class ParametrosAspect {
```

@Pointcut .-

- Se utiliza para definir un punto de corte.
- Toma el nombre del método que está anotando. Realmente este método actúa como un marcador por lo cual está vacío.

```
@Pointcut ("execution(* getPedido*(..)) && args(codPedido,..)")  
private void getPedido(String codPedido) {}
```

@Before .-

- Anota un método para que se ejecute antes el consejo que el método.

```
@Before("getPedido(codPedido)")
public void registraPedido(JoinPoint j, String codPedido) {
    System.out.println(" ParametrosAspect metodo::" +
        j.getSignature().getName());
    System.out.println("## ParametrosAspect Before1 ParamAspect :: "
        + "Se ha solicitado un pedido ##" + codPedido);
}
```

@After .-

- El consejo se ejecuta después del método.
- **@After**; da igual si hay errores o no en el método.
- **@AfterReturning**; se ejecuta si el metodo finaliza correctamente sin errores.
- **@AfterThrowing**; se ejecuta si el método devuelve un error.

@AfterReturning, @AfterThrowing .-

```
@AfterThrowing(throwing="ex",pointcut="getPedido(codPedido)")
public void tratarError(Exception ex, String codPedido){
    System.out.println(" ParametrosAspect ERROR::" +
        ex.getMessage());
    System.out.println("ParametrosAspect PEDIDO ERRONEO::" +
        codPedido);
}

@AfterReturning(pointcut="getPedido(codPedido)")
public void devuelvePedido(String codPedido) {
    System.out.println("## ParametrosAspect ultimo1 ParamAspect:: "
        + "Se ha solicitado un pedido ##"+codPedido);
}
```

@Around .-

- Sirve para anotar el consejo que se ejecutará antes y después del método.

```
@Around("getPedido(codPedido)")
public void decide(ProceedingJoinPoint jp, String codPedido)
    throws Throwable {
    System.out.println("## Around ANTES:: "
        + "Se ha solicitado un pedido ##" +
        jp.getTarget().getClass());
    System.out.println("CODIGO ANTES: " + codPedido);
    Object ret = jp.proceed();
    System.out.println("CODIGO::" + codPedido);
    System.out.println("## Around DESPUES:: "
        + "Se ha solicitado un pedido ##" + ret);
}
```

Argumentos con anotaciones .-

```
@Pointcut ("execution(* getPedido*(..)) && args(codPedido) ")  
private void getPedido(String codPedido) {}  
  
@Around("getPedido(codPedido) ")  
public void decide(ProceedingJoinPoint jp, String codPedido)  
    throws Throwable {
```

Uso de Spring para acceso a Bases de datos

Tema 4

Spring JDBC .-

- Este modulo de Spring trabaja en la capa de persistencia.
- Permite acceder a la BBDD a través del estándar JDBC, como también posibilita la integración con otros framework de persistencia como Hibernate, Ibatis, JPA, ...etc.

Recursos necesarios .-

- Necesitamos los siguientes componentes:
 - **DataSource**; contiene los datos de la conexión.
 - **Template**; plantilla para la reutilización de código.
 - **Mapeador**; clase que transforma el ResultSet en la instancia del modelo.
 - **Dao**; clase que recoge todos los métodos de acceso a la BBDD.

DataSource .-

- Se declara en el archivo XML como un bean de una de estas clases:
- **DriverManagerDataSource**; esta clase me proporciona una conexión nueva cada vez.
- **SingleConnectionDataSource**; actúa como un Singleton, esto es, siempre devuelve la misma conexión.

Crear un DataSource .-

```
<bean id="dataSource"
      class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName" value="com.mysql.jdbc.Driver" />
  <property name="url" value="jdbc:mysql://localhost/curso_spring" />
  <property name="username" value="root" />
  <property name="password" value="root" />
</bean>
```

Template .-

- Spring proporciona 3 tipos de plantilla:
 - **JdbcTemplate**; permite acceso a JDBC. Es la plantilla más básica.
 - **NamedParametersJdbcTemplate**; Se utiliza para parámetros con nombre.
 - **SimpleJdbcTemplate**; Recoge las novedades de java 5 como por ejemplo los tipos genéricos

Crear la plantilla .-

```
<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource" />
</bean>

<bean id="namedTemplate"
    class="org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate">
    <constructor-arg ref="dataSource" />
</bean>

<bean id="jdbcTemplate"
    class="org.springframework.jdbc.core.simple.SimpleJdbcTemplate">
    <constructor-arg ref="dataSource" />
</bean>
```

Crear el mapeador .-

```
public class ProductoMapeador implements RowMapper<Producto> {  
  
    public Producto mapRow(ResultSet rs, int rowNum) throws SQLException {  
        Producto producto = new Producto();  
        producto.setCodigo(rs.getInt("codigo"));  
        producto.setDescripcion(rs.getString("descripcion"));  
  
        return producto;  
    }  
}
```

```
<bean id="mapeador" class="es.prac.spring.ProductoMapeador" />
```

Crear el DAO .-

```
public class DaoProducto {  
  
    private JdbcTemplate jdbcTemplate;  
    private RowMapper<Producto> mapeador;  
  
    public List<Producto> findAllProductos() {  
        String sql = "select codigo, descripcion from productos";  
        return (List<Producto>) (jdbcTemplate.query(sql, mapeador));  
    }  
  
    public void insertarProducto(int codigo, String descripcion) {  
        jdbcTemplate.update("insert into productos values(" + codigo +  
            ",'" + descripcion + "')");  
    }  
}
```

Configurar el DAO .-

```
<bean id="daoProducto" class="es.prac.spring.DaoProducto">  
    <property name="jdbcTemplate" ref="jdbcTemplate" />  
    <property name="mapeador" ref="mapeador" />  
</bean>
```

DAO con patrón Factory Method .-

```
<bean id="daoProducto" class="datos.DAOFactory"
      factory-method="getProductoDAO">
  <property name="jdbcTemplate" ref="namedTemplate" />
  <property name="mapeador" ref="mapeador" />
</bean>
```

Acceso a datos con JPA

- JPA es el acrónimo de Java Persistence API y se podría considerar como el estándar de los frameworks de persistencia.
- En JPA utilizamos anotaciones como medio de configuración.
- Consideramos una entidad al objeto que vamos a persistir o recuperar de una base de datos. Se puede ver una entidad como la representación de un registro de la tabla.
- Toda entidad ha de cumplir con los siguientes requisitos:
 - Debe implementar la interface Serializable
 - Ha de tener un constructor sin argumentos y este ha de ser público.
 - Todas las propiedades deben tener sus métodos de acceso `get()` y `set()`.
- Para crear una entidad utilizamos la anotación `@Entity`, con ella marcamos un POJO como entidad.

Acceso a datos con JPA

- Vamos a trabajar con una base de datos en memoria H2, para ello necesitamos agregar la siguiente dependencia al pom.xml

```
<dependency>  
  <groupId>com.h2database</groupId>  
  <artifactId>h2</artifactId>  
</dependency>
```

Acceso a datos con JPA

- Spring nos facilita el trabajar con los datos incluyendo estas dependencias:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-rest</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Acceso a datos con JPA

- Debemos mapear la entidad a manejar en la base de datos:

```
@Entity
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private long id;

    private String descripcion;
    private double precio;
```

Acceso a datos con JPA

- Y por ultimo tener el repositorio donde se generaran las queries de forma automatica;

```
@RepositoryRestResource(collectionResourceRel = "productos", path = "productos")
public interface ProductoRepository extends PagingAndSortingRepository<Producto, Long> {

    List<Producto> findByDescripcion(@Param("descripcion") String descripcion);

}
```

Acceso a datos con JPA

- Una vez levantada la aplicación con Spring Boot ya podemos probarla con los siguientes comandos en consola:

```
// comandos a ejecutar en consola "con permiso de administrador"
// Acceso al servicio
// curl http://localhost:8080

// Consultar todos los productos
// curl http://localhost:8080/productos

// Alta de producto y consulta para su verificacion
// curl -i -X POST -H "Content-Type:application/json" -d '{"id": 1, "descripcion": "Macarrones", "precio":0.87}' http://localhost:8080/productos
// curl http://localhost:8080/productos

// Búsqueda de un producto por su id y luego por su descripcion
// curl http://localhost:8080/productos/1
// curl http://localhost:8080/productos/search/findByDescripcion?descripcion=Macarrones

// Modificar precio del producto y consulta para su verificacion
// curl -X PUT -H "Content-Type:application/json" -d '{"id": 1, "descripcion": "Macarrones", "precio":0.98}' http://localhost:8080/productos/1
// curl http://localhost:8080/productos/1

// Borrar un producto y consulta para su verificacion
// curl -X DELETE http://localhost:8080/productos/1
// curl http://localhost:8080/productos
```

Control de transacciones en Spring

Tema 5

Definición de transacción .-

- Una transacción se define como un conjunto de operaciones que o bien se ejecutan todas o no se ejecuta ninguna.

Propiedades ACID .-

- Atomicidad; todas las operaciones se ejecutan como un todo.
- Consistencia; Si el sistema es consistente antes de la transacción, debe seguir siéndolo al finalizar esta.
- Aislamiento; Nadie puede acceder a los datos involucrados en una transacción mientras esta esté en marcha.
- Durabilidad; Los datos de la transacción tras efectuar el commit, deben ser permanentes.

TransactionManager .-

- Puede ser:
 - DataSourceTransactionManager
 - HibernateTransactionManager
 - JpaTransactionManager

```
<bean id="transactionManager"  
      class="org.springframework.jdbc.datasource.DataSourceTransactionManager">  
    <property name="dataSource" ref="dataSource" />  
</bean>
```

Transacciones en Spring .-

- Tenemos 3 formas de crear una transacción:
 1. Programativa
 2. Declarativa sin anotaciones
 3. Declarativa con anotaciones

1. Programativa .-

- Para el modelo programativo necesitamos declarar un TransactionTemplate.

```
<bean id="transactionTemplate"  
      class="org.springframework.transaction.support.TransactionTemplate">  
    <property name="transactionManager" ref="transactionManager" />  
</bean>
```

- En el dao inyectamos el TransactionTemplate como propiedad:

```
<bean id="daoProducto" class="datos.DAOFactory"  
      factory-method="getProductoDAO">  
    <property name="jdbcTemplate" ref="jdbcNamedTemplate" />  
    <property name="mapeador" ref="mapeador" />  
    <property name="transactionTemplate" ref="transactionTemplate" />  
</bean>
```

Utilización en el dao .-

- Todo lo que hagamos dentro del método doInTransaction estará en la misma transacción.

```
transactionTemplate.execute(new TransactionCallbackWithoutResult() {  
  
    protected void doInTransactionWithoutResult(TransactionStatus ts) {  
        try {  
            Iterator<Producto> it = lista.iterator();  
            while (it.hasNext()) {  
                Producto p = it.next();  
                insertarProducto(p);  
            }  
        } catch (Exception ex) {  
            ts.setRollbackOnly();  
            ex.printStackTrace();  
        }  
    }  
});
```

2. Declarativa sin anotaciones .-

- Es necesario incluir los espacios de nombres:

```
xmlns:aop="http://www.springframework.org/schema/aop"  
xmlns:tx="http://www.springframework.org/schema/tx"
```

- No hay que definir un TransactionTemplate
- Utiliza AOP, para ello es necesario 2 nuevos componentes:
 - <tx:advice>
 - <aop:config>

Continuación .-

```
<tx:advice id="txAdvice">
  <tx:attributes>
    <tx:method name="altaProductos" propagation="REQUIRED"
              isolation="DEFAULT" />
    <tx:method name="*" propagation="SUPPORTS" read-only="true" />
  </tx:attributes>
</tx:advice>

<aop:config>
  <aop:advisor pointcut="execution(* *..ServicioProductos.*(..))"
              advice-ref="txAdvice" />
</aop:config>
```

OJO!! .- 2 puntos delante del nombre de la clase

Modelos de propagación .-

- MANDATORY; el método debe ejecutarse dentro de una transacción. Si no existe, se genera una excepción.
- NESTED; el método debe ejecutarse dentro de una transacción anidada, si no existe se genera una nueva.
- NEVER; No debe existir una transacción, si la hubiera genera excepción.
- NOT_SUPPORTED; No debe ejecutarse dentro de una transacción, si existiese esta se quedaría suspendida.
- REQUIRED; el método debe ejecutarse dentro de una transacción. Si no existe, se genera una nueva.
- REQUIRES_NEW; el método debe ejecutarse dentro de una transacción nueva. Si ya existe, esta se quedaría suspendida.
- SUPPORTS; Soporta transacciones, si existe se ejecutará en este contexto, si no , no pasa nada.

Modelos de aislamiento .-

- DEFAULT; Nivel de aislamiento predeterminado de la BBDD.
- READ_UNCOMMITTED; Puede leer cambios que aun no se han aplicado. Es peligroso ya que puede efectuar lecturas sucias.
- READ_COMMITTED; permite las lecturas de transacciones simultaneas que se han aplicado. Puede darse lectura fantasma y no repetible
- REPEATABLE_READ; varias lecturas del mismo campo generan el mismo resultado a menos que la tx los modifique. Se puede producir lectura fantasma
- SERIALIZABLE; Cumple con el principio ACID.

3. Declarativa con anotaciones .-

- Es necesario incluir los espacios de nombres:

```
xmlns:aop="http://www.springframework.org/schema/aop"  
xmlns:tx="http://www.springframework.org/schema/tx"
```

- Este elemento indica a Spring que examine todos los beans o métodos anotados como @Transactional

```
<tx:annotation-driven />
```

Continuación .-

```
@Transactional
public void altaProductos(List<Producto> listaProductos) {
    System.out.println("altaProductos");
    Iterator<Producto> it = listaProductos.iterator();
    while (it.hasNext()) {
        Producto p = it.next();
        daoProducto.insertarProducto(p);
    }
}
```

Fundamentos Desarrollo Java con Spring Framework



Completa nuestra encuesta
de satisfacción a través del QR

Curso Fundamentos desarrollo
Java con Spring Framework (8-18
junio)



GRACIAS

indra